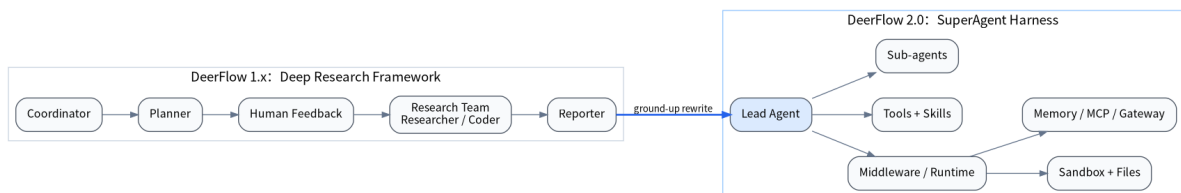


ByteDance DeerFlow 2.0

从 Deep Research Framework 到 Long-horizon SuperAgent Harness

架构演化 • 源码机制 • Research Policy • Context Engineering • 可复用 Know-Why / Know-How



研究日期：2026-08-13

主要依据：ByteDance 官方 GitHub 主仓库、main-1.x 分支、Backend Architecture / Configuration / API、Skills、CHANGELOG 与源码说明

执行摘要

核心判断

DeerFlow 2.0 的本质不是“更复杂的 Deep Research Graph”，而是把研究、编码、数据处理、内容生成等任务统一放进一个可扩展 Agent Harness。Deep Research 从固定主流程变成 Skill；执行可靠性则下沉到 Lead Agent、middleware、sandbox、subagent、memory、tools/MCP、run state 等基础设施。

截至 2026-08-13，GitHub API 显示 bytedance/deer-flow 约 79,890 stars、10,930 forks，仓库最近一次 push 为 2026-08-12，主语言 Python，MIT License。[S1] 这些数字只用于说明社区规模和当前活跃度，不代表系统质量。

DeerFlow 1.x 的官方定位是 community-driven Deep Research framework，核心是把语言模型与搜索、抓取、Python 执行等专用工具组合起来；当时典型架构被描述为 Coordinator → Planner → Human Feedback → Research Team（Researcher/Coder）→ Reporter。[S9][S10]

DeerFlow 2.0 的 README 明确写明：它是 ground-up rewrite，与 v1 不共享代码；官方将它重新定义为 open-source super agent harness，内置 filesystem、memory、skills、sandbox-aware execution，以及按需规划和生成 sub-agents 的能力。[S2]

这一变化意味着工程重心从“设计一张任务图”转移到“设计一个可靠的 Agent Runtime”。固定 Graph 负责确定流程；Harness 负责提供可组合的能力、状态、隔离、并发、策略与治理，让模型在运行时决定具体路径。

维度	DeerFlow 1.x	DeerFlow 2.0
产品定位	Deep Research Framework	Long-horizon SuperAgent Harness
主控制方式	预定义研究 workflow / 多角色图	Lead Agent + middleware + tools + dynamic subagents
Deep Research	系统主任务	Skills 中的一种能力 / research policy
执行环境	搜索、爬取、Python 工具	统一 Sandbox + 文件系统 + artifact 交付
上下文策略	主要围绕研究流程组织	subagent 隔离 + summarization + filesystem + progressive skills
扩展面	Researcher/Coder 等角色	MCP、Skills、自定义工具、自定义 Agent/Subagent
工程关注点	研究流程正确性	长任务可靠性、隔离、权限、状态、并发、可观测性

本报告重点回答两个问题：第一，为什么 DeerFlow 会从 Deep Research Framework 演化为 Harness；第二，这个演化里有哪些可以迁移到其他 Coding Agent / Research Agent / Enterprise Agent 的通用 know-how。

目录与阅读建议

1. 研究范围、证据与时间线
2. DeerFlow 1.x: 固定 Deep Research Graph 的逻辑
3. 为什么从 Research Framework 重写成 Harness
4. DeerFlow 2.0 总体架构
5. Lead Agent 与 Middleware: 把可靠性从 Prompt 下沉到 Runtime
6. Agent Loop 与请求生命周期
7. Sub-agents: Context Isolation、并发与 Fan-out/Fan-in
8. Sandbox / Filesystem / Artifacts: 让 Agent 真正 “有电脑”
9. Context Engineering: 压缩、外置状态、渐进加载
10. Memory: 跨 Session 的持久上下文
11. Skills: 从 Prompt 模板到可版本化能力模块
12. Deep Research Skill 与 GitHub Deep Research Skill
13. Tools / MCP / Custom Agents
14. Guardrails、Loop Detection、授权与生产级约束
15. Deployment / API / Observability
16. DeerFlow 2.0 做 Deep Research 时究竟发生什么
17. 架构优点、代价与潜在边界
18. 可积累的 Know-Why / Know-How
19. 如何借鉴 DeerFlow 构建自己的 Agent Runtime
20. 研究结论与源码阅读路线

快速阅读: 如果目标是理解架构演化, 优先读第 2-5、16、18 章; 如果目标是自己实现 Agent Runtime, 优先读第 5-15、19 章; 如果目标是 Deep Research 方法论, 优先读第 2、11、12、16 章。

1. 研究范围、证据与时间线

1.1 研究范围

本报告聚焦 ByteDance 官方开源项目 bytedance/deer-flow，核心对象是当前 main 分支的 DeerFlow 2.0，同时回看 main-1.x 的 Deep Research Framework，用于解释架构为何发生断裂式变化。报告不把社区 Fork、第三方教程或二手文章当作主要证据。

- 事实层：README、CHANGELOG、Backend README、Architecture、Configuration、API、Skills 与 GitHub API 元数据。
- 实现层：Lead Agent、middleware、subagent、sandbox、skills 等代码路径与官方仓库内部 AGENTS/CLAUDE 指南。
- 推断层：基于上述事实解释“为什么这么设计”“有哪些工程权衡”。推断会明确标注为分析，不冒充官方结论。

1.2 关键时间线

时间	事件	意义
2025-05-07	GitHub 仓库创建（API metadata）[S1]	DeerFlow 开源项目起点
1.x 时期	官方定位为 Deep Research framework [S9]	围绕搜索、抓取、代码执行和研究报告建立固定流程
2026-02-28	README 称 DeerFlow 2 发布，并登上 GitHub Trending #1 [S2]	公开切换到 SuperAgent Harness 叙事
2026-06-15	CHANGELOG 记录 2.0.0，声明 ground-up rewrite [S8]	稳定版本节点；与 v1 无代码共享
2026-06-25	GitHub Discussion 宣布 DeerFlow 2.0.0 release [S8]	正式社区发布公告
2026-08-12	GitHub API 显示最近 push [S1]	截至本报告仍持续开发

时间线说明

README 的“2 月 28 日发布 version 2”与 CHANGELOG / Release 的“6 月 2.0.0”并不矛盾：可理解为 2.0 方向公开发布与后续稳定 2.0.0 里程碑的不同节点。报告将两者分开表述。

2. DeerFlow 1.x：固定 Deep Research Graph 的逻辑

DeerFlow 1.x 的官方 README 将其定义为 Deep Research framework，目标是把 LLM 与 web search、crawling、Python code execution 等专用工具组合起来。[S9] 在当时的架构讨论中，典型流程被描述为：Coordinator → Planner → Human Feedback → Research Team → Reporter，其中 Research Team 内含 Researcher/Coder。[S10]

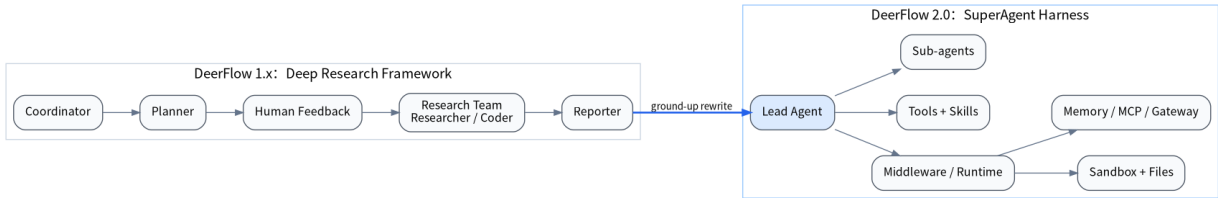


图 2-1 DeerFlow 从 1.x Deep Research Graph 到 2.0 Harness 的结构变化

2.1 为什么这种 Graph 在 Deep Research 中合理

- 研究任务天然包含阶段：澄清问题、规划子问题、搜集证据、必要的计算/代码、最后报告。固定 Graph 能把这些阶段显式化。
- 角色分离降低单一模型上下文混杂：Researcher 专注信息搜集，Coder 专注计算，Reporter 专注整合。
- 固定节点便于插入 Human Feedback，也便于复现某个研究阶段的状态。
- 对产品早期而言，明确工作流比“让模型自由规划一切”更可控。

2.2 固定 Graph 的结构性限制

固定 Graph 的优势也是限制：它把“任务是什么”预先写进系统拓扑。只要用户需求偏离“研究→报告”，就需要扩展节点、分支或重新定义状态。这会造成能力与 workflow 紧耦合。

限制	表现	后果
任务类型被图结构绑定	研究流程天然适配，网页、PPT、数据管道等需要额外节点	Graph 逐渐膨胀
角色粒度预定义	Researcher/Coder/Reporter 是固定角色	遇到新领域时角色复用不够灵活
上下文和状态常随图传播	节点越多、历史越长，state 越重	长任务容易触及 token / state 复杂度
执行能力是“节点的工具”	工具和流程耦合	难形成通用运行时
扩展以改图为主	新增能力需要改 workflow	社区二次开发成本上升

3. 为什么从 Research Framework 重写成 Harness

DeerFlow 2.0 README 给出的直接原因是：社区把 DeerFlow 用在研究之外，包括数据管道、slide deck、dashboard、内容自动化等；团队由此判断 DeerFlow 更像“给 Agent 提供工作基础设施的 harness”，而不只是 research tool。[S2]

架构转折

v1 的中心问题是“研究流程怎么走”；v2 的中心问题变成“无论任务怎么走，Agent 都需要哪些稳定基础设施”。

3.1 从 Workflow Engineering 到 Runtime Engineering

Workflow Engineering	Runtime / Harness Engineering
把任务拆成固定节点	提供可组合能力，让模型动态决定路径
节点负责状态和工具	middleware / tool runtime 负责横切能力
流程图是产品核心	Lead Agent + execution environment 是核心
新增任务 = 新流程	新增任务 = 新 Skill / Tool / Subagent 配置
Research 是系统本体	Research 是一种可加载能力

这是一种“控制反转”：系统不再试图穷举每种任务的图，而是把稳定约束固化到 runtime，把任务知识放入 Skills，把并行执行放入 Subagents，把环境能力放入 Sandbox，把跨会话信息放入 Memory。

3.2 Ground-up rewrite 的意义

官方明确说明 2.0 与 v1 不共享代码。[S2][S8] 这意味着迁移不是局部重构，而是抽象层改变：从 research-specific graph 抽象到 general-purpose harness。工程上只有当旧抽象无法自然承载新需求时，才值得付出这种重写成本。

4. DeerFlow 2.0 总体架构

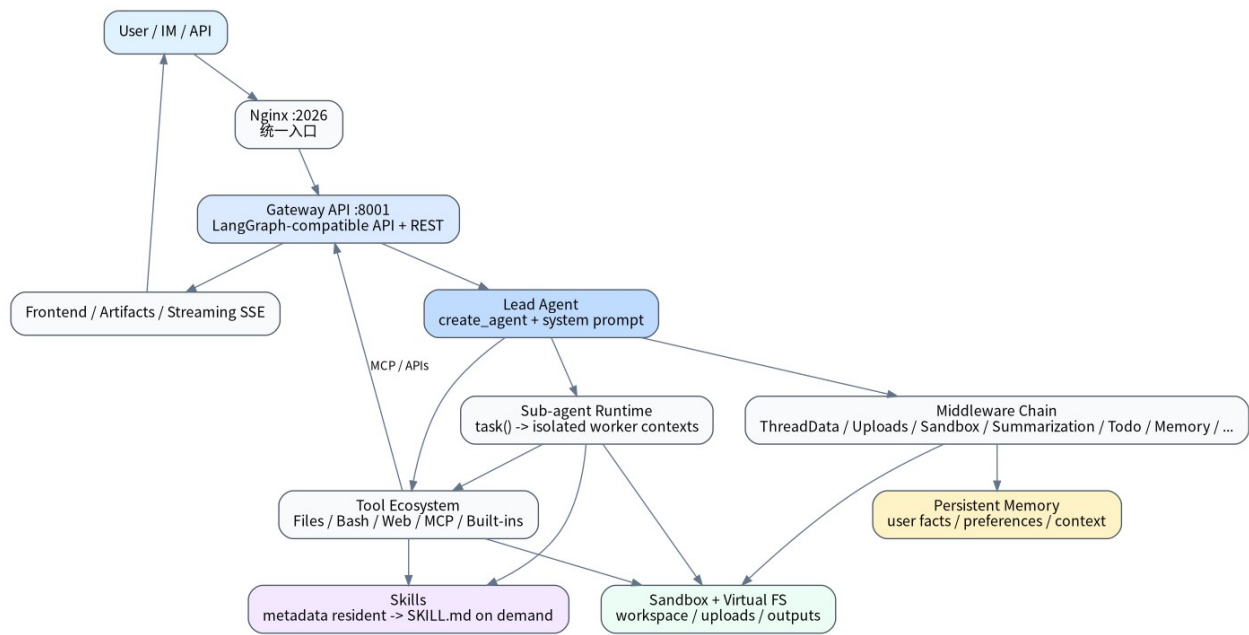


图 4-1 DeerFlow 2.0 总体组件关系（依据官方 Backend README / Architecture 文档重绘）

后端采用 LangGraph-based super agent。Nginx 提供统一入口；Gateway API 同时承载 LangGraph-compatible agent interactions 与 models、MCP、skills、memory、uploads、artifacts 等 REST 能力；单一 lead_agent 是 runtime entry point。[S3][S4]

层	主要职责	关键价值
Gateway / API	线程、run、streaming、模型、MCP、Skills、Memory、Uploads、Artifacts	把 Agent runtime 产品化
Lead Agent	模型调用、系统提示、工具选择、subagent delegation	统一决策入口
Middleware	线程目录、sandbox、summarization、todo、memory、限制与错误处理	把横切可靠性从 Prompt 下沉
Tools / MCP	Web、文件、bash、内置工具、第三方工具	可替换的能力面
Subagents	异步委派、并发执行、独立上下文	扩展认知与执行吞吐
Sandbox / FS	隔离执行、虚拟路径、workspace/uploads/outputs	可审计的“工作电脑”
Skills	工作流、最佳实践、参考资料，按需加载	把领域 know-how 模块化
Memory	跨 session user context / facts / preferences	长期个性化与持续工作

最值得注意的是：2.0 没有把 “Planner、Researcher、Reporter” 继续当作顶层固定节点。它改用 Lead Agent + task() delegation。规划能力可通过 plan mode/TODO middleware 按请求开启；研究行为则通过 Skills 和工具完成。[S12][S6]

5. Lead Agent 与 Middleware：把可靠性从 Prompt 下沉到 Runtime

官方 Backend README 将 `lead_agent` 描述为单一 LangGraph runtime entry point，由动态模型选择、middleware chain、tool system、subagent delegation、以及注入 skills/memory/working-directory guidance 的 system prompt 组成。[S3]

5.1 为什么是单一 Lead Agent

- 减少“谁是最终控制者”的不确定性：所有工具、subagent 与状态最终回到同一决策入口。
- 允许任务拓扑在运行时动态形成，而不是由静态 Graph 预先穷举。
- 更适合 Skills 模式：Skill 影响的是 Agent 的行为策略，不需要新建 Graph 节点。
- 更容易统一权限、token、loop、tool error、observability 等 runtime policy。

5.2 Middleware 是 DeerFlow 2.0 最值得借鉴的机制之一

Backend README 列出的核心 middleware 包括 ThreadData、Uploads、Sandbox、Summarization、TodoList、Title、Memory、ViewImage、Clarification；当前仓库内部指南还描述了更多生产级 middleware，例如 SubagentLimit、LoopDetection、token/finish-reason 与 tool error handling 等。[S3][S11]

Middleware 类别	解决的问题	为什么不只写在 Prompt
Thread / Files	每线程工作目录与路径隔离	Prompt 无法真正创建隔离目录
Sandbox	安全取得执行环境	安全边界必须由代码保证
Summarization	接近 token limit 时压缩上下文	不能依赖模型“自觉少说”
Todo / Plan	维护多步任务状态	需要持久 state，而非文本约定
Subagent Limit	限制单轮/单 run 委派数量	防止 runaway parallelism
Loop Detection	重复工具调用自动收敛	模型可能无法识别自己陷入 loop
Clarification	把需要用户输入的状态转成 interrupt	需要 runtime 控制流语义
Memory	异步提取并注入持久事实	跨 session 需要外部存储

Know-why

Prompt 擅长表达“应该怎么做”；middleware 擅长保证“必须这样做”。只要一个约束涉及安全、资源、状态、并发或生命周期，就应该优先考虑 runtime enforcement，而不是只写 Prompt。

6. Agent Loop 与请求生命周期

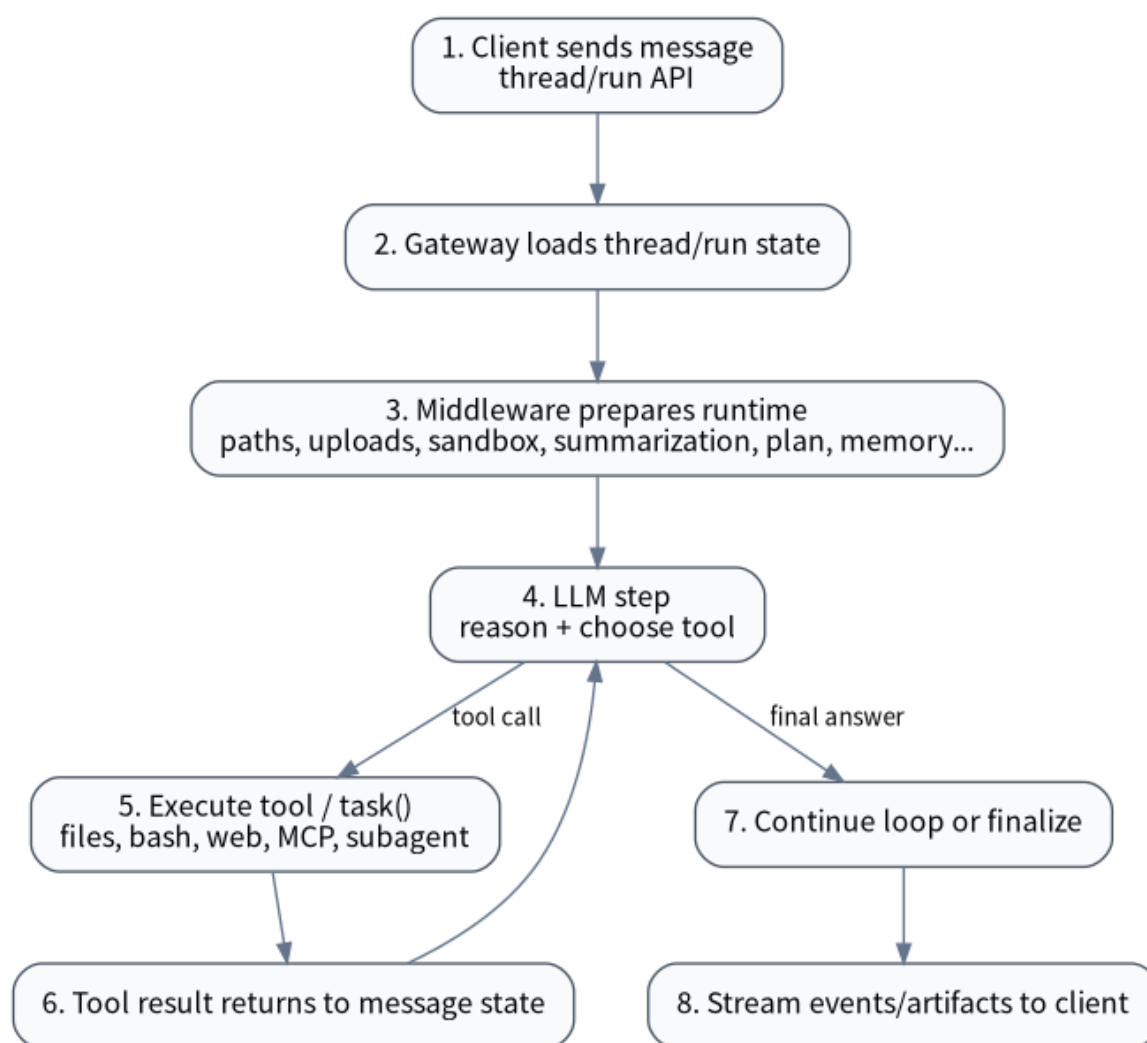


图 6-1 从请求进入到工具循环与流式返回的典型生命周期

Architecture 文档给出的请求流是：Client → Nginx → Gateway；Gateway 加载/创建 thread state，执行 middleware，随后模型处理消息并可调用 bash/web 等工具，工具结果回写 messages，最终通过 SSE 流式返回。[S4]

6.1 本质仍然是 Agent Loop

无论外层组件多复杂，核心仍可简化为：模型读取当前 state → 产生 tool call 或 final answer → runtime 执行 tool → 将结果写回 state → 再次调用模型。DeerFlow 2.0 的价值不在于发明了不同的循环，而在于把循环周围的“生产基础设施”系统化。

最小心智模型

LLM 决策循环不等于 Agent 系统。真正的 Agent Runtime = Loop + State + Tools + Policies + Execution Environment + Lifecycle + Observability。

6.2 Plan Mode 不是强制 Planner 节点

Plan mode 通过运行时配置 `is_plan_mode` 动态启用 Todo middleware，并向 Agent 增加 `write_todos` 工具；它是一个可选能力，而不是所有任务必经的 Planner 节点。[S12] 这就是 v2 的典型设计：把结构化规划从“固定拓扑”改成“按需 runtime feature”。

7. Sub-agents: Context Isolation、并发与 Fan-out/Fan-in

DeerFlow 2.0 的 lead agent 可以通过 `task()` 工具动态委派任务。Backend README 描述内置 general-purpose 与 bash subagent，后台执行、状态跟踪与 SSE 事件；当前内部指南说明 per-response 并发限制与 per-run 总量限制由 SubagentLimitMiddleware enforcement。[S3][S11]

7.1 Subagent 的真正价值不是“多一个角色”

更重要的是上下文隔离。README 说明每个 sub-agent 有独立 scoped context；研究任务可 fan-out 到多个方向，再由 lead agent 汇总。[S2] 这使 subagent 成为一种 Context Engineering 原语：把细节工作从主上下文移走。

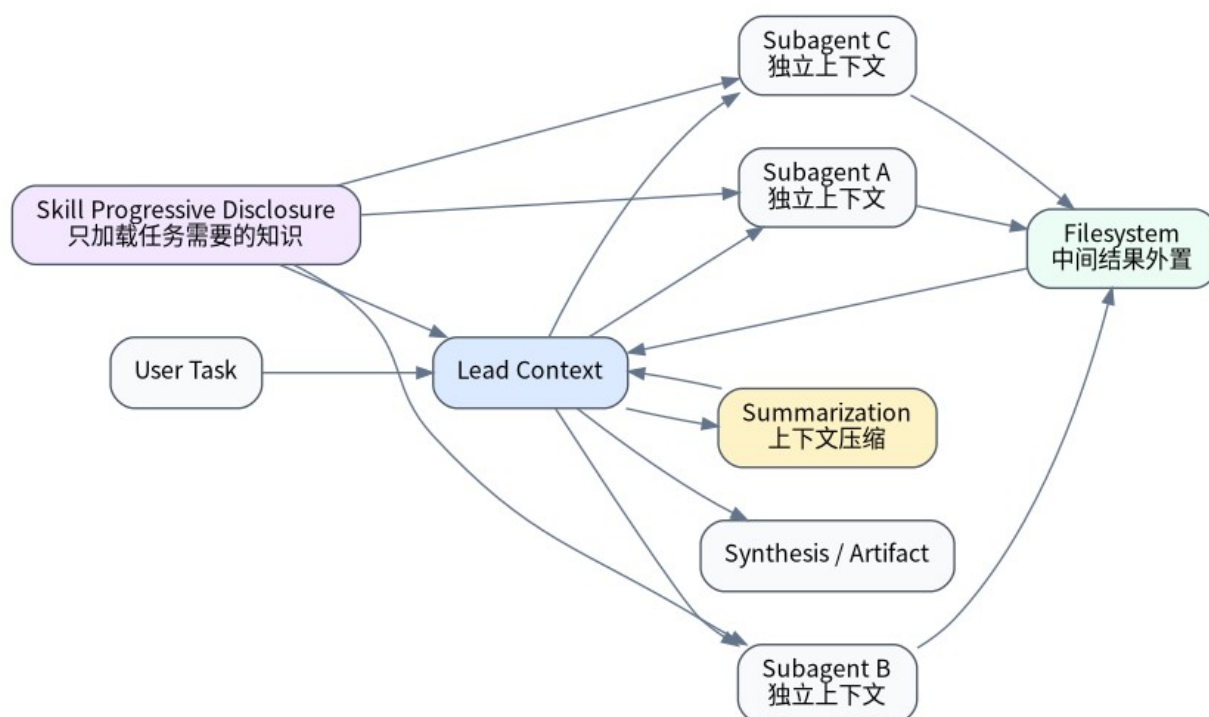


图 7-1 Subagent 隔离 + 文件系统 + summarization 构成的上下文工程

收益	机制	适用场景
降低上下文污染	子任务只看相关上下文	论文批量抽取、代码模块分析、网页多方向研究
提高并行吞吐	多个 subagent 同时工作	互不依赖的研究分支
把中间 token 留在子上下文	主 Agent 只接收结构化结果	长任务/大文档处理
支持专用配置	工具/skills/model 可不同	便宜模型做抽取，强模型做 synthesis
失败隔离	单个子任务失败不必破坏全局状态	高不确定外部工具任务

7.2 为什么需要并发上限与总量上限

无限 fan-out 会引发 API 成本、sandbox 数量、线程池、外部搜索配额和汇总上下文同时爆炸。当前官方内部指南描述：单轮并发限制被 clamp，且按 run 还存在总委派 budget，达到上限后 middleware 会阻止继续 task calls，让 lead agent 基于已有结果收敛。[S11]

Know-how

Subagent 系统至少要有四层预算：单轮并发数、单 run 总任务数、单 subagent timeout/max-turns、全局资源预算。只做“支持并发”而不做预算，往往会把 Agent 变成成本放大器。

8. Sandbox / Filesystem / Artifacts: 让 Agent 真正 “有电脑”

DeerFlow 将 /mnt/user-data/workspace、uploads、outputs 作为稳定虚拟路径，并把 /mnt/skills 映射到 Skills 目录。Backend Architecture 与 README 都强调 per-thread isolation。[S3][S4]

虚拟路径	语义	工程意义
/mnt/user-data/uploads	用户输入文件	只读/受控输入边界
/mnt/user-data/workspace	Agent 中间工作区	外置中间状态与临时文件
/mnt/user-data/outputs	最终交付物	明确 artifact contract
/mnt/skills	Skills 内容	领域知识与执行指南的运行时挂载

8.1 Local Sandbox 与 Container Sandbox

Configuration 文档明确提醒：LocalSandboxProvider 是宿主机便利模式，不是安全的 shell isolation boundary，因此 host bash 默认关闭；需要 shell 时优先使用 Docker-based AioSandboxProvider。还支持 provisioner/Kubernetes 方式。[S5]

这个细节非常重要：不要把“路径映射”误认为“安全隔离”。文件工具能映射到每线程目录，不代表 shell 已被安全沙箱化。DeerFlow 把二者区分开，是正确的安全建模。

8.2 Filesystem 是长任务的外部工作记忆

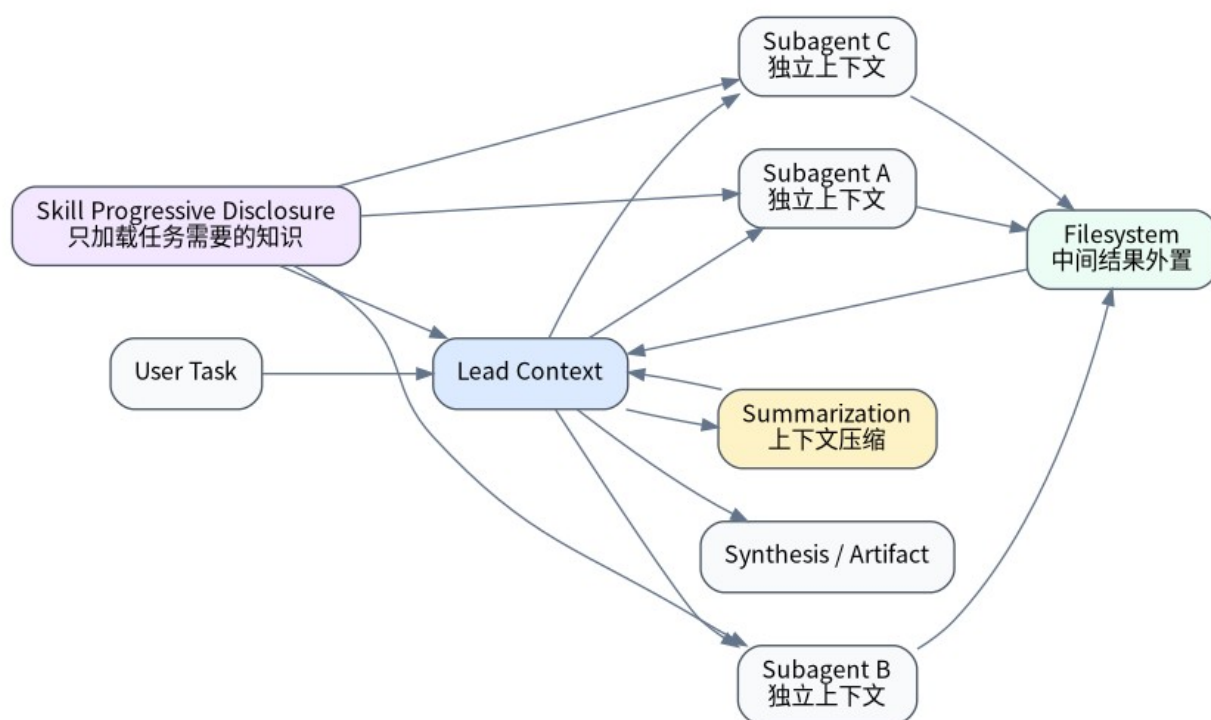
对长任务而言，文件系统不仅用于产出文件，也是上下文卸载机制：搜索结果、数据表、代码、草稿、阶段总结可以落盘，主模型不必把所有中间数据持续保留在 context window。

Know-why

Context window 应该保存 “下一步决策真正需要的信息”；可复查但不必常驻的信息，应尽量外置到 filesystem / store，并在需要时再读回。

9. Context Engineering：压缩、外置状态、渐进加载

DeerFlow README 直接把 Context Engineering 作为核心能力：subagent context isolation、session 内 summarization、把中间结果转存到文件系统、压缩暂时不重要的信息。[S2] Skills 也采用 progressive loading，只在任务需要时加载完整 Skill 内容。[S2]



9.1 四种互补策略

1. 隔离 (isolate)：把独立子任务交给 subagents，避免主上下文被细节占满。
2. 压缩 (compress)：对历史与已完成阶段生成摘要，减少 token 占用。
3. 外置 (externalize)：把中间结果、代码、数据、artifact 放到文件系统。
4. 渐进披露 (progressive disclosure)：Skills 只常驻 metadata，需要时读取 SKILL.md 和附属资源。

这四种策略对应四种不同的 token 浪费来源：不相关任务、过长历史、大体积数据、未使用知识。把它们混为一种“摘要”会丢失架构清晰度。

9.2 Context Engineering 与 Prompt Engineering 的边界

Prompt Engineering	Context Engineering
优化某段指令的措辞	决定每次模型调用到底看到什么
关心 instruction quality	关心 state composition / retrieval / compression
通常是局部文本问题	是运行时信息架构问题
难控制长期 token 增长	通过隔离/外置/压缩直接控制 token

10. Memory：跨 Session 的持久上下文

Backend README 将 Memory System 描述为 LLM-powered persistent context retention：自动分析对话中的 user context、facts、preferences，以结构化形式保存，并把 top facts + context 注入 system prompt；当前默认文档还描述 JSON file storage 与 mtime cache invalidation。[S3]

10.1 Memory 与 Thread State 不是一回事

概念	生命周期	典型内容
Message/Thread State	一次 thread / run	对话消息、工具结果、todo、run state
Workspace Files	任务或 thread	中间文件、代码、数据、artifact
Persistent Memory	跨 session / 用户级	长期偏好、背景、稳定事实
Skill Knowledge	跨用户/版本化	通用流程、最佳实践、领域 SOP

把这几类信息混在一个“memory”概念里会导致错误：例如，项目临时细节不应进入用户长期记忆；通用 SOP 也不应该靠记忆抽取生成，而应该进入 Skill。

10.2 Memory 的工程风险

- 错误事实会跨会话放大，因此需要 confidence、scope 和 contradiction handling。
- 隐式注入 memory 会影响模型行为，应能审计“本次到底注入了哪份 memory identity”。Backend README 提到 run-level memory identity event。[S3]
- 本地 JSON 适合单机开发，但大型多 worker 部署需要特别关注一致性、并发写、迁移与治理。这里属于工程推论，不代表官方当前生产部署只有 JSON。

11. Skills：从 Prompt 模板到可版本化能力模块

DeerFlow 将标准 Agent Skill 定义为结构化 capability module，通常包含 SKILL.md，其中写 workflow、最佳实践和参考资源。官方 README 说明 Skills 按需渐进加载；Configuration 说明 skills 位于 public/custom 目录并自动发现；API 可列出、启停和安装 .skill 包。[S2][S5][S15]

11.1 Skill 的设计价值

传统 System Prompt	Agent Skill
所有规则长期常驻	metadata 常驻，正文按需加载
难以模块化版本	一个目录/文件就是一个能力单元
组织知识与 runtime 紧耦合	可独立维护、替换、安装
不同 Agent 难复用	Lead/Subagent 可按配置筛选 Skills
Prompt 越来越长	渐进披露控制 token

当前仓库的 Skill System 还支持 per-agent filtering：null 表示继承全局 enabled skills，[] 可禁用全部，显式列表可只加载指定 skills。[S5] 这使“能力边界”可以成为 Agent 配置，而不仅是 Prompt 约定。

11.2 Progressive Disclosure 的机制

README 表述为“只有任务需要时才加载”；一个关于 slash skill activation 的官方 RFC 更明确描述了当前模式：system prompt 暴露 enabled skill 的 name、description、location，由模型在需要时 read_file 读取 SKILL.md；完整正文与引用资源不是常驻上下文。[S14]

Know-how

Skill metadata 要足够让模型做“是否加载”的决策，但不能把完整教程塞进 description；description 是路由信号，不是正文。

12. Deep Research Skill 与 GitHub Deep Research Skill

DeerFlow 2.0 最能证明 “Research 已经变成 Skill” 的证据，是官方仓库直接提供 deep-research 与 github-deep-research 两套 Skill。[S6][S7]

12.1 Deep Research Skill：把研究方法论编码成 Policy



图 12-1 Deep Research Skill 的多轮研究策略抽象

Deep Research Skill 的核心原则不是特定搜索 API，而是“单次搜索不够”。它要求从多个角度 broad exploration，阅读重要来源全文，识别知识缺口并迭代 refined queries，同时覆盖正面、挑战、局限和时效性，直到满足质量标准后再生成内容。[S6]

Research Policy	对应风险
至少从多个角度探索	防止单 query 造成视角偏差
重要来源读全文	防止只根据 snippet 误判
识别 gaps 后继续检索	防止一次检索即停止
关注反例/挑战/限制	防止 confirmation bias
加入 temporal qualifier	防止用过时信息回答当前问题
研究完成后再写内容	防止“边搜边定结论”过早收敛

12.2 GitHub Deep Research Skill：Research Policy 的垂直化

GitHub Deep Research Skill 将研究拆成 GitHub API → discovery → deep investigation → deep dive 等轮次，强调 broad-to-narrow query strategy、官方来源优先、commit/PR/issue/release 等仓库证据，并设计 confidence scoring 与 structured report 输出。[S7]

这说明 Skills 不只是“提示词模板”，而可以编码一种领域研究协议：数据源优先级、搜索顺序、验证方法、置信度、输出结构和工具使用约束。

关键转变

DeerFlow 1.x: Research = Graph topology. DeerFlow 2.0: Research = Skill policy + generic agent runtime + tools + subagents.

13. Tools / MCP / Custom Agents

DeerFlow 的工具生态包括 sandbox 文件工具、built-in tools、community web/search/scraping integrations，以及任意 MCP server；MCP 支持 stdio、SSE、HTTP，并有缓存和配置动态更新机制。[S3]

13.1 为什么 MCP 放在 Harness 层而不是 Skill 层

Skill 主要描述“怎么做”；Tool/MCP 提供“能做什么”。把二者分离后，同一个 research skill 可以切换不同搜索/抓取工具，同一个 MCP tool 也可以被不同 skill 复用。

层	应包含什么	不应包含什么
Skill	流程、策略、最佳实践、工具选择原则	长期连接状态、密钥、执行隔离
Tool/MCP	能力接口、参数、执行结果	完整领域 SOP
Middleware	权限、预算、错误处理、生命周期	具体业务知识
Sandbox	执行边界、文件/命令能力	任务推理策略

13.2 Custom Agents / Subagents

config.example.yaml 展示了 custom subagent 配置：可以定义 description、system_prompt、tools whitelist、skills whitelist、model、max_turns、timeout 等；默认可继承 parent model，也可指定更便宜/更合适的模型。[S17]

这带来 Model Routing 的自然落点：主 Agent 用强模型负责规划与综合；批量抽取、bash 或领域子任务可以使用不同模型，前提是结果协议足够稳定。

14. Guardrails、Loop Detection、授权与生产级约束

长任务 Agent 的失败通常不是“模型答错一句”，而是 runaway loops、过度工具调用、错误权限、无限并发、sandbox 泄漏、不可取消 run 等系统性问题。DeerFlow 2.0 的当前仓库已经把多类约束放进 runtime。

14.1 Loop Detection 与 Subagent Limit

内部 backend guide 描述 LoopDetectionMiddleware 会检测重复工具调用并在 hard stop 时强制模型回到最终文本；SubagentLimitMiddleware 同时限制每轮并发与每 run 总委派量。[S11] 这是典型的“外部终止条件”，不能只依赖模型自律。

14.2 Guardrails

官方 Guardrails 文档提供 GuardrailMiddleware 与 provider abstraction，可在 tool call 前允许/拒绝，并配置 fail-closed 行为。[S13] 这类机制适合企业环境中的工具 allowlist/denylist 或更复杂的策略服务。

14.3 Authorization 与运行时治理

当前 Lead Agent 源码包含模型与工具授权相关逻辑，强调 runtime 选择的模型也必须经过 action-scoped authorization；这说明 v2 已经在从“单用户开源 Agent”向“多用户、可治理平台”扩展。[S18]

Know-why

越接近“Agent 平台”，越不能把安全与资源约束写成自然语言。必须有结构化 policy decision + runtime enforcement + audit event。

15. Deployment / API / Observability

Backend API 同时提供 LangGraph-compatible endpoints 与 Gateway REST。客户端可以管理 thread/run、streaming，同时通过 API 管理 models、MCP、skills、uploads、artifacts、memory 等。
[S15]

15.1 为什么 Gateway 很重要

- 把 Agent 从“Python 函数”升级为可被 Web、CLI、IM channel、其他 Agent 调用的服务。
- 统一 thread/run 生命周期，便于 cancel、resume、stream 和审计。
- 把 artifacts、uploads、skills、memory 这些非消息资源纳入同一控制面。
- 允许外部工具或 Claude Code 通过 HTTP 调用 DeerFlow，而不必嵌入内部代码。

15.2 Observability 的研究价值

一个 Deep Research Agent 的质量不能只看 final report。需要记录：搜索 query、source、subagent task、tool call、token、artifact changes、loop stop reason、memory injection identity 等。

DeerFlow 当前仓库已经出现 run event、subagent status、token usage、tracing 等机制。

对于自己的 Agent Runtime，建议把“最终答案 eval”与“trajectory eval”分开：前者看结果质量，后者看搜索/工具/成本/中止是否合理。

16. DeerFlow 2.0 做 Deep Research 时究竟发生什么

把前面的组件合起来，可以更准确地理解 DeerFlow 2.0 的 Deep Research：它不是启动一个固定 “Research Graph”，而是 Lead Agent 在通用 runtime 中识别 research task，按需读取 Deep Research Skill，调用 web/search/fetch/MCP 工具，并在复杂任务中使用 task() 把不同研究分支委派给独立 subagents。阶段结果可落盘，长历史由 summarization 压缩，最后由 Lead Agent 综合并生成 report/artifact。

阶段	可能使用的机制	核心目的
任务理解	Lead Agent + Skill metadata	判断是否需要 deep-research skill
研究策略加载	read SKILL.md (progressive disclosure)	加载多轮研究方法，而非长期常驻
问题展开	plan/todo + model reasoning	形成多个研究方向
并行研究	task() subagents	上下文隔离 + fan-out
信息获取	web search/fetch / MCP / files	获取一手/权威证据
中间持久化	workspace files	外置大体积中间状态
上下文控制	summarization	避免长 run context 爆炸
终止控制	loop/subagent/token policies	防止无限执行
最终综合	Lead Agent + report skill/outputs	形成带来源的交付物

最重要的抽象

Deep Research = Research Policy × Generic Agent Loop × Search/Fetch Tools × Context Engineering × Evidence/Artifact State × Runtime Guardrails。

17. 架构优点、代价与潜在边界

17.1 优点

- 任务泛化：同一 harness 可研究、编码、数据分析、生成网页/PPT，而不是为每类任务维护一张大 Graph。
- 能力组合：Skills 与 Tools 解耦，MCP 与自定义工具可替换。
- 长任务可持续性：subagent isolation、summarization、filesystem 降低主上下文压力。
- 执行真实性：sandbox + 文件系统让 Agent 能生成可交付 artifacts，而不仅是文本。
- 平台化：Gateway、thread/run、memory、skills management、IM channels 形成完整运行面。
- 治理可插拔：middleware 适合持续增加 guardrail、authorization、budget、logging。

17.2 代价

代价	原因	应对
系统复杂度显著上升	runtime、API、sandbox、storage、middleware、subagents 都要维护	强模块边界 + contract tests
动态行为更难复现	Lead Agent 路径不是固定 Graph	event journal + deterministic configs + eval
并发放大成本	subagents + web + sandbox	预算/timeout/total cap
Skill routing 依赖模型判断	progressive disclosure 需要正确激活	metadata eval + 显式 activation 机制
Memory 可能污染行为	错误长期事实被持续注入	scope/confidence/audit/delete
安全面变宽	bash、MCP、uploads、artifacts 都是攻击面	sandbox + authz + guardrails + secret isolation

17.3 不应过度学习的地方

不是所有 Agent 都需要 DeerFlow 级别的 Harness。单一内部流程、短任务、无文件执行、不需要跨会话 memory 的场景，固定 Graph 或简单 Loop 可能更可靠、更便宜。架构应由任务熵、执行时长、工具数量、并发需求和安全边界决定。

18. 可积累的 Know-Why / Know-How

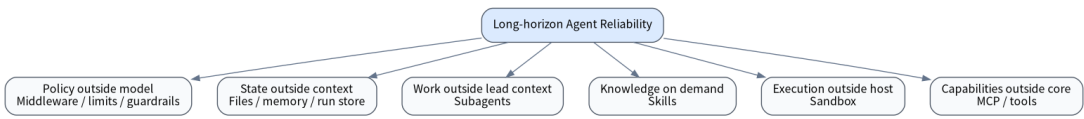


图 18-1 DeerFlow 2.0 可以抽象出的六条长任务可靠性原则

18.1 Know-Why：为什么 Harness 比“更大的 Prompt”重要

1. 模型不是可靠的资源调度器：并发、timeout、budget、loop 需要 runtime 保底。
2. Context Window 不是数据库：大数据、中间文件、历史结果应外置。
3. 领域方法论不应全部常驻：Skills 用 progressive disclosure 把知识按需加载。
4. 复杂任务不是单线程推理问题：独立子问题可用 subagents 隔离并行。
5. 工具执行不是聊天：真正的文件、代码和网络操作需要 sandbox 与 lifecycle。
6. 研究质量不是“搜了多少次”：关键是 evidence coverage、source quality、gap-driven iteration 和停止条件。

18.2 Know-How：可直接迁移的实现原则

原则	实现建议
Policy / Capability 分离	Policy 放 Skill, Capability 放 Tool/MCP, Enforcement 放 Middleware
状态分层	messages、thread state、workspace、persistent memory、skills 分开存
Subagent 契约	输入只给必要背景，输出固定 schema/summary + evidence pointers
Context budget	每种信息指定 resident / summarized / externalized / retrieved 策略
Execution budget	并发、总任务数、tool calls、timeout、token、cost 都有上限
Artifact contract	明确 uploads/workspace/outputs，不让最终文件散落
Research provenance	claim → source → retrieval timestamp → confidence
可观测性	记录 run、tool、subagent、token、stop reason、workspace changes
安全默认值	本地非隔离 shell 默认关闭；敏感工具必须显式授权
Eval 驱动 Skill	对 skill activation、tool selection、citation correctness 做独立评测

19. 如何借鉴 DeerFlow 构建自己的 Agent Runtime

下面不是复刻 DeerFlow 的源码，而是抽象一条更适合从 MVP 演进的路线。

19.1 Phase 0: 最小 Agent Loop

先实现 model → tool call → execute → tool_result → model 的闭环，只保留 3-5 个工具。成功标准是工具 contract、错误返回和 state serialization 稳定。

19.2 Phase 1: State + Filesystem

- 每 thread 独立 workspace/uploads/outputs。
- tool result 大于阈值时写文件，只回传摘要与路径。
- 所有 artifact 通过 outputs contract 交付。

19.3 Phase 2: Middleware / Policy

- 加入 tool error handling、token budget、loop detection、permission/guardrails。
- 不要先做十几个 Agent 角色，先把单 Agent 的 runtime 做稳。

19.4 Phase 3: Skills

- 把重复出现的领域 SOP 从 system prompt 拆成 Skills。
- metadata 做 routing；完整正文按需加载。
- 给 Skill 做 activation eval，而不是只看最终答案。

19.5 Phase 4: Subagents

- 只对“上下文独立、可并行、结果可结构化”的任务启用。
- 强制并发/总量/timeout/max-turn budgets。
- 子 Agent 默认不能递归无限 spawn。

19.6 Phase 5: Memory + Gateway + Platformization

- 先定义 memory scope 与删除/审计，再做自动提取。
- 当需要多客户端、stream、cancel、resume、IM 接入时再引入 Gateway/RunStore。

建议的演进顺序

Loop → State/FS → Middleware/Guardrails → Skills → Subagents → Memory/Gateway。这个顺序能避免在 Agent 核心尚不稳定时过早进入多 Agent 和平台复杂度。

20. 研究结论与源码阅读路线

DeerFlow 最值得研究的不是某个“搜索 Prompt”，而是它展示了开源 Agent 工程的一次抽象升级：从“把任务画成图”升级到“为模型提供长任务运行环境”。Deep Research 仍然重要，但已经不再绑死系统拓扑，而被表达成可加载的 Research Policy。

如果把 DeerFlow 2.0 的设计压缩成一句话：****让模型负责不确定的决策，让 runtime 负责确定的约束；让 context 保存决策信息，让 filesystem/store 保存大体积状态；让 Skills 承载可复用 know-how，让 Subagents 承载可隔离工作。****

20.1 推荐源码阅读顺序

1. README_zh.md / README.md：先理解产品抽象从 Deep Research → Harness。[S2]
2. backend/README.md：理解 Lead Agent、middleware、sandbox、subagent、memory、tools。[S3]
3. backend/docs/ARCHITECTURE.md：理解 API request flow 与虚拟路径。[S4]
4. backend/packages/harness/deerflow/agents/lead_agent/agent.py：理解 agent factory 与 runtime option。[S18]
5. backend/AGENTS.md：理解最新 middleware/subagent 约束与模块边界。[S11]
6. config.example.yaml：理解 sandbox、subagents、custom agents、models 的配置面。[S17]
7. skills/public/deep-research/SKILL.md：理解 Research Policy 如何从 graph 独立出来。[S6]
8. skills/public/github-deep-research/SKILL.md：理解垂直 research protocol。[S7]
9. backend/docs/GUARDRAILS.md：理解 runtime policy enforcement。[S13]
10. main-1.x README + historical architecture issue：对照旧的 Deep Research workflow。[S9][S10]

附录 A：关键源码与文档路径

路径	重点
README.md / README_zh.md	2.0 产品定位、核心特性、v1/v2 分界
backend/README.md	后端架构总览：Lead Agent、middleware、sandbox、subagent、memory、tools
backend/docs/ARCHITECTURE.md	request flow、virtual path mapping、安全边界
backend/docs/CONFIGURATION.md	sandbox、skills、模型、subagent 等配置
backend/docs/API.md	Gateway + LangGraph-compatible API
backend/packages/harness/deerflow/agents/lead_agent/agent.py	Lead Agent factory / runtime options / middleware assembly
backend/packages/harness/deerflow/agents/middlewares/	Summarization、Todo、Memory、Loop、Limits、Errors 等
backend/packages/harness/deerflow/subagents/	subagent executor 与配置
backend/packages/harness/deerflow/sandbox/	Sandbox 抽象与 Local provider
backend/packages/harness/deerflow/community/	AIO sandbox、Web 工具等 community integrations
backend/packages/harness/deerflow/skills/	Skill loading / metadata / security
skills/public/deep-research/SKILL.md	通用 Deep Research Methodology
skills/public/github-deep-research/SKILL.md	GitHub 深度研究协议
skills/public/systematic-literature-review/SKILL.md	subagent 批量论文抽取的实践示例
config.example.yaml	可运行配置与 custom subagent 示例
CHANGELOG.md	2.0.0 及后续 runtime hardening 变更
main-1.x/README.md	原始 Deep Research Framework

附录 B：来源、证据等级与引用索引

证据等级：A = 官方主仓库当前文档/API/源码；B = 官方历史分支或官方 Issue/RFC（用于历史/设计讨论）；“分析” = 本报告基于 A/B 证据做出的工程推论。

索引	等级	来源	URL	用途
S1	A	GitHub REST repository metadata	https://api.github.com/repos/bytedance/deer-flow	2026-08-13 获取：stars、forks、license、updated/pushed 时间等。
S2	A	README.md / README_zh.md (main)	https://github.com/bytedance/deer-flow	2.0 定位、ground-up rewrite、Subagents、Sandbox、Context Engineering、Skills。
S3	A	backend/README.md	https://github.com/bytedance/deer-flow/blob/main/backend/README.md	Lead Agent、middleware、sandbox、subagent、memory、tool ecosystem、Gateway。
S4	A	backend/docs/ARCHITECTURE.md	https://github.com/bytedance/deer-flow/blob/main/backend/docs/ARCHITECTURE.md	request flow、virtual paths、安全。
S5	A	backend/docs/CONFIGURATION.md	https://github.com/bytedance/deer-flow/blob/main/backend/docs/CONFIGURATION.md	Local/Docker/K8s sandbox、Skills filtering、GitHub token。
S6	A	skills/public/deep-research/SKILL.md	https://github.com/bytedance/deer-flow/blob/main/skills/public/deep-research/SKILL.md	多角度、多轮、gap-driven、source quality 的研究协议。
S7	A	skills/public/github-deep-research/SKILL.md	https://github.com/bytedance/deer-flow/blob/main/skills/public/github-deep-research/SKILL.md	GitHub API→Discovery→Deep Investigation→Deep Dive、置信度与报告结构。
S8	A	CHANGELOG.md / Releases / Discussion #3795	https://github.com/bytedance/deer-flow/blob/main/CHANGELOG.md	2.0.0 时间、ground-up rewrite、runtime hardening。
S9	B	main-1.x README.md	https://github.com/	v1 Deep Research

			bytedance/deer-flow/blob/main-1.x/README.md	Framework 定位与工具生态。
S10	B	Issue #270	https://github.com/bytedance/deer-flow/issues/270	历史架构描述：Coordinator→Planner→Human Feedback→Research Team→Reporter。
S11	A	backend/AGENTS.md	https://github.com/bytedance/deer-flow/blob/main/backend/AGENTS.md	当前 middleware chain、SubagentLimit、LoopDetection、并发/总量约束。
S12	A	backend/docs/plan_mode_usage.md	https://github.com/bytedance/deer-flow/blob/main/backend/docs/plan_mode_usage.md	is_plan_mode 与 Todo middleware 的按请求启用。
S13	A	backend/docs/GUARDRAILS.md	https://github.com/bytedance/deer-flow/blob/main/backend/docs/GUARDRAILS.md	GuardrailMiddleware、provider、fail-closed。
S14	B/A	Issue #3017 Slash Skill activation RFC	https://github.com/bytedance/deer-flow/issues/3017	RFC 为拟议功能；其中对当前 progressive disclosure 行为的描述用于理解现状。
S15	A	backend/docs/API.md	https://github.com/bytedance/deer-flow/blob/main/backend/docs/API.md	LangGraph-compatible + Gateway API、skills/memory/uploads/artifacts。
S17	A	config.example.yaml	https://github.com/bytedance/deer-flow/blob/main/config.example.yaml	sandbox/subagent/custom agent/model 配置示例。
S18	A	backend/packages/harness/deerflow/agents/lead_agent/agent.py	https://github.com/bytedance/deer-flow/blob/main/backend/packages/harness/deerflow/agents/lead_agent/agent.py	Lead Agent factory、runtime options、authorization 等实现线索。

研究方法与置信度说明

本报告优先使用官方主仓库当前 main 的文档与源码；历史结构使用 main-1.x 与官方 Issue 作为辅助。对于“为什么这么设计”“适合什么场景”等内容，本报告基于代码结构做工程分析，并未声称这些推论代表 ByteDance 官方表态。

时间敏感数字（stars、forks、push 时间）均以 2026-08-13 获取的 GitHub API 为截面，之后可能变化。

— End of Report —